

Traffic Demand Prediction

Rithanya Raj & Anjan Mahapatra

June 5, 2026

Abstract

This document provides a comprehensive, elaborated mathematical and technical reference for the Traffic Demand Prediction (TDP) v8 Aggressive model. The model employs a multi-stage pipeline combining sophisticated feature engineering, target encoding, ensemble learning, and post-processing calibration. Traffic demand forecasting is a critical component of intelligent transportation systems, enabling resource allocation, congestion mitigation, and dynamic routing optimization. This document details all mathematical formulations, loss functions, evaluation metrics, and implementation considerations across nine base models unified through Optuna-optimized weighting and multi-seed bagging. The “aggressive” designation refers to the extensive feature set, aggressive regularisation tuning, and deep ensemble stacking used to maximize predictive accuracy while controlling overfitting. This elaborated version includes additional derivations, numerical examples, expanded theoretical background, and **explicit rationales** for every design choice, explaining why each component was selected over alternatives.

Contents

1	Introduction & Problem Formulation	6
1.1	Problem Statement	6
1.1.1	Formal Problem Definition	6
1.2	Motivation for Demand Prediction	6
1.3	Model Architecture Overview	7
2	Data Preprocessing & Temporal Feature Extraction	8
2.1	Data Quality & Validation	8
2.1.1	Missing Value Assessment	8
2.1.2	Outlier Detection (IQR Method)	8
2.2	Timestamp Parsing & Temporal Decomposition	8
2.2.1	Hour Extraction	8
2.2.2	Minute Extraction	8
2.2.3	Continuous Time-of-Day Feature	9
2.2.4	15-Minute Time Slot Discretization	9
2.3	Day-Level Temporal Features	9
2.3.1	Day-of-Week Extraction	9
2.3.2	Day-of-Year Extraction	10
3	Feature Engineering	11
3.1	Harmonic (Fourier) Time Features	11
3.1.1	Mathematical Foundation: Fourier Series	11
3.1.2	k -th Order Sine Temporal Feature	11
3.1.3	k -th Order Cosine Temporal Feature	12
3.2	Calendar & Seasonal Features	12
3.2.1	Day-of-Week Encoding	12
3.2.2	Weekend Binary Indicator	12
3.2.3	Day-of-Year Sine Harmonic	13
3.2.4	Day-of-Year Cosine Harmonic	13
3.3	Peak Period Binary Flags	13
3.3.1	Morning Peak (07:00–09:00)	13
3.3.2	Evening Peak (17:00–19:00)	13
3.3.3	Noon Peak (11:00–13:00)	13
3.3.4	Off-Peak (00:00–05:00)	13
3.3.5	Morning Period (05:00–10:00)	14
3.3.6	Afternoon Period (12:00–17:00)	14
3.3.7	Evening Period (18:00–23:00)	14
3.3.8	Night Period (00:00–04:00)	14
3.4	Geospatial Features & Clustering	14
3.4.1	Geohash Decoding	14
3.4.2	Geographic Clustering via K-Means	15
3.4.3	Cluster Assignment Feature	15
3.5	Interaction Features	15
3.5.1	Cluster-Hour Interaction	15
3.5.2	Cluster-DayOfWeek Interaction	16
3.5.3	Latitude-Time Interaction	16
3.5.4	Longitude-Time Interaction	16

3.5.5	Hour-TimeSlot Interaction	16
3.6	Temporal Lag Features	16
3.6.1	Day-48 Demand Lag	16
3.7	Road Network & Weather Features	17
3.7.1	Temperature Imputation Strategy	17
3.7.2	Temperature Missing Indicator	17
3.7.3	Large Vehicles Allowed Binary	17
3.7.4	Landmarks Present Binary	17
3.7.5	High Capacity Road Indicator	17
4	Target Encoding (Fold-Safe Smoothing)	19
4.1	Theoretical Foundation: Empirical Bayes Smoothing	19
4.1.1	Likelihood & Prior	19
4.1.2	Posterior Distribution (via Bayes' Theorem)	19
4.2	General Smoothing Formula	19
4.2.1	Smoothed Target Encoding	19
4.2.2	Fold-Safe Application Protocol	20
4.3	Target Encoding Applications	20
4.3.1	Geohash Target Encoding ($m = 10$)	20
4.3.2	Geohash Standard Deviation	20
4.3.3	Geohash-TimeSlot Target Encoding ($m = 5$)	21
4.3.4	RoadType Target Encoding ($m = 20$)	21
4.3.5	Geographic Cluster Target Encoding ($m = 15$)	21
4.3.6	Hour-of-Day Target Encoding ($m = 25$)	21
5	Model Training Objectives & Loss Functions	22
5.1	Regression Loss Functions	22
5.1.1	Mean Squared Error (MSE) Loss	22
5.1.2	Root Mean Squared Error (RMSE)	22
5.2	LightGBM Regularisation	22
5.2.1	LightGBM Objective with L2 & L1 Regularisation	22
5.3	XGBoost Objective	23
5.3.1	XGBoost MSE Loss with Regularisation	23
5.4	CatBoost Objective	24
5.4.1	CatBoost RMSE Loss with Regularisation	24
5.5	ExtraTrees Objective	24
5.5.1	Extremely Randomized Trees MSE Split Criterion	24
5.6	HistGradientBoosting Objective	25
5.6.1	Histogram-based Gradient Boosting	25
6	Evaluation Metrics	26
6.1	Coefficient of Determination (R^2)	26
6.1.1	R^2 Score Formula	26
6.2	Mean Absolute Error (MAE)	26
6.2.1	MAE Formula	26
6.3	Out-of-Fold (OOF) R^2	27
6.3.1	Cross-Validation R^2 Formula	27
6.4	Per-Fold R^2	27
6.4.1	R^2 Per Fold	27

7	Ensemble & Stacking Methods	28
7.1	Ensemble Theory: Bias-Variance Decomposition	28
7.1.1	Error Decomposition	28
7.1.2	Ensemble Error (Uncorrelated Models)	28
7.2	Optuna Weight Optimisation	28
7.2.1	Weighted Ensemble Optimisation Objective	28
7.2.2	Optuna Search Strategy	29
7.2.3	Optuna Ensemble Prediction	29
7.3	Multi-Seed Bagging	29
7.3.1	10-Seed Bagged Ensemble	29
7.4	Final Weighted Blend	30
7.4.1	Weighted Combination: 70% Optuna + 30% Bagging	30
8	Post-Processing & Calibration	31
8.1	Clipping to Valid Range	31
8.1.1	Demand Clipping to $[0, 1]$	31
8.2	Scaling Calibration	31
8.2.1	Mean-Based Calibration Scaling Factor	31
8.2.2	Calibrated Final Prediction	32
8.3	Uncertainty Quantification	32
8.3.1	Prediction Interval via Bagging Variance	32
8.3.2	Confidence Interval (95%)	32
9	Cross-Validation Setup	33
9.1	K-Fold Cross-Validation Strategy	33
9.1.1	K-Fold Stratified Split	33
9.1.2	Temporal Split (Time-Series Aware)	33
9.2	Stratification Considerations	33
9.2.1	Demand Quantile Stratification	33
10	Feature Set Definitions	35
10.1	Feature Categories	35
10.1.1	Numeric Base Features (28 total)	35
10.1.2	Categorical Features (5 total)	35
10.1.3	Target-Encoded Features (6 total)	35
10.2	Feature Set Definitions	36
10.2.1	LGB Feature Set (LightGBM-specific)	36
10.2.2	Target-Encoded Feature Set	36
11	Data Leakage Prevention	37
11.1	Fold-Safe Target Encoding Protocol	37
11.1.1	Leakage Prevention During Target Encoding	37
11.2	Day-48 Lag Masking	37
11.2.1	Prevent Data Leakage for Lag Feature	37
11.3	Hyperparameter Validation	37
11.3.1	No Hyperparameter Tuning on Test Data	37
11.3.2	Cross-Validation Boundaries	38

12 Uncertainty Quantification & Error Analysis	39
12.1 Residual Analysis	39
12.1.1 Residual Definition	39
12.1.2 Mean Residual (Bias)	39
12.1.3 Residual Standard Deviation	39
12.2 Segment-Wise Performance	39
12.2.1 R^2 by Peak Period	39
12.2.2 R^2 by Geographic Cluster	40
13 Implementation Best Practices	41
13.1 Numerical Stability	41
13.1.1 Feature Normalization	41
13.1.2 Handling NaN and Infinite Values	41
13.2 Reproducibility	41
13.2.1 Random Seed Management	41
13.3 Model Serialization	41
14 Advanced Topics	42
14.1 Concept Drift Detection	42
14.1.1 Performance Degradation Monitoring	42
14.2 Hyperparameter Sensitivity Analysis	42
14.2.1 Parameter Importance via Gradients	42
A Detailed Mathematical Derivations	43
A.1 Derivation of the Target Encoding Smoothing Formula	43
A.2 Derivation of Ensemble Variance Reduction	43
A.3 Derivation of the Calibration Scale Factor	43
B Summary Table: All Formulas	43
C Hyperparameter Reference	44
D Debugging Checklist	44
E Glossary	44

1 Introduction & Problem Formulation

1.1 Problem Statement

The traffic demand prediction problem is formulated as a supervised regression task. Given a set of contextual features extracted from spatio-temporal, meteorological, and road network data, the objective is to predict normalized traffic demand $\hat{y} \in [0, 1]$ at specific geographic locations and temporal intervals.

1.1.1 Formal Problem Definition

$$\min_{\hat{y}} \mathcal{L}(y, \hat{y}(X)) \quad (1)$$

where:

- $X \in \mathbb{R}^{n \times p}$ = feature matrix with n samples and p features
- $y \in [0, 1]^n$ = normalized target (traffic demand)
- $\hat{y} : \mathbb{R}^p \rightarrow [0, 1]$ = prediction function (model)
- $\mathcal{L}(y, \hat{y})$ = loss function quantifying prediction error

Selection Rationale for Normalization: Demand counts vary widely across locations and times. Normalizing to $[0, 1]$ ensures that all models operate on a consistent scale, which improves numerical stability and allows fair comparison of errors across different geographies. The maximum observed demand is used as the normalizer to preserve the relative magnitude.

1.2 Motivation for Demand Prediction

Accurate traffic demand prediction enables:

1. **Resource Optimization:** Allocation of vehicles, drivers, and infrastructure based on predicted demand
2. **Congestion Mitigation:** Proactive routing and capacity planning
3. **Revenue Forecasting:** Demand-based pricing and surge estimation
4. **Operational Efficiency:** Reduction of idle time and improved utilization
5. **Environmental Impact:** Optimized fleet deployment reduces carbon footprint
6. **Safety:** Anticipating high-demand periods allows pre-positioning of emergency services

Selection Rationale for These Use Cases: These five areas represent the primary stakeholders' needs identified through business requirements analysis. Resource optimization directly impacts operational costs; congestion mitigation is a regulatory priority; revenue forecasting drives financial planning; operational efficiency affects driver satisfaction; environmental impact aligns with sustainability goals. Safety was added as an emerging benefit.

1.3 Model Architecture Overview

The TDP v8 Aggressive pipeline consists of five primary stages:

$$\begin{array}{ccccccc} \text{Raw Data} & \xrightarrow{\text{Preprocessing}} & \text{Feature Matrix} & \xrightarrow{\text{Encoding}} & \text{Enriched Features} & & \\ & & & & & \xrightarrow{\text{Models}} & \text{Base Predictions} \xrightarrow{\text{Ensemble}} \hat{y}_{\text{final}} \end{array} \quad (2)$$

Key Components:

- 28 numeric base features (temporal, geographic, interaction)
- 5 categorical features (road attributes, weather)
- 6 target-encoded features with fold-safe smoothing
- 9 base models (LightGBM variants, XGBoost, CatBoost, ExtraTrees, HistGradientBoosting)
- 2-layer ensemble: Optuna weighting + multi-seed bagging
- Calibration and post-processing

Selection Rationale for Five-Stage Pipeline: The pipeline separates concerns: preprocessing cleans data, feature engineering creates signal, encoding handles high-cardinality variables, models provide diverse perspectives, and ensemble combines them robustly. This modularity simplifies debugging and allows independent upgrades.

Why “Aggressive”? The v8 Aggressive model pushes the boundaries of feature engineering and ensemble complexity: it includes high-order harmonic terms (up to 3rd order), 20 geographic clusters, 8 peak period flags, and 6 target-encoded features. The ensemble combines 9 diverse models with a sophisticated blending strategy, and hyperparameters are aggressively tuned using Optuna with a wide search space. This approach achieves state-of-the-art accuracy but requires careful cross-validation and regularisation to avoid overfitting.

2 Data Preprocessing & Temporal Feature Extraction

2.1 Data Quality & Validation

Before feature engineering, the raw dataset undergoes validation to ensure integrity.

2.1.1 Missing Value Assessment

$$\text{missingness_rate}_j = \frac{\#\{\text{null values in column } j\}}{n} \times 100\% \quad (3)$$

Selection Rationale: Assessing missingness rate per column helps decide whether to impute, drop, or create missing indicators. Columns with $> 80\%$ missing are dropped to avoid adding noise. This threshold is standard in industry practice.

2.1.2 Outlier Detection (IQR Method)

$$\text{outlier}_i = \mathbf{1}(x_i < Q_1 - 1.5 \cdot \text{IQR} \text{ or } x_i > Q_3 + 1.5 \cdot \text{IQR}) \quad (4)$$

where Q_1, Q_3 are first and third quartiles, and $\text{IQR} = Q_3 - Q_1$.

Selection Rationale: IQR is chosen over Z-score because demand data is not Gaussian. The $1.5 \times \text{IQR}$ rule is a well-established non-parametric outlier threshold. Outliers are winsorized (capped) rather than removed because extreme demand events (e.g., concerts, emergencies) are real and should not be discarded; capping limits their undue influence on tree splits.

2.2 Timestamp Parsing & Temporal Decomposition

Timestamps are parsed from ISO 8601 format and decomposed into constituent temporal components. Each component captures different periodicities in demand patterns.

2.2.1 Hour Extraction

$$h = \left\lfloor \frac{\text{timestamp_in_seconds}}{3600} \right\rfloor \bmod 24, \quad h \in \{0, 1, \dots, 23\} \quad (5)$$

Purpose: Extract hourly component; captures morning peak (07:00–09:00), afternoon (12:00–17:00), evening peak (17:00–19:00), and off-peak periods.

Rationale: Demand exhibits strong hourly periodicity correlated with commute patterns, meal times, and business hours.

Selection Rationale: Hour is extracted as an integer from 0–23. This discrete representation allows tree models to split on specific hours (e.g., hour=8 vs hour=20) without imposing linearity, unlike a continuous time feature which would force monotonic relationships.

2.2.2 Minute Extraction

$$m = \text{timestamp_in_seconds} \bmod 3600 \div 60, \quad m \in \{0, 1, \dots, 59\} \quad (6)$$

Purpose: Extract minute component for fine-grained temporal resolution.

Rationale: Within-hour demand variations can be significant in high-frequency settings; enables 15-minute slot discretization.

Selection Rationale: Minute precision is retained because demand can change rapidly (e.g., after a concert ends). However, using minutes directly would create 1440 categories, which is too sparse. Thus we aggregate to 15-minute slots.

2.2.3 Continuous Time-of-Day Feature

$$t_{\min} = 60h + m \in [0, 1440), \quad t_{\min} \in \mathbb{R} \quad (7)$$

Purpose: Convert time into a continuous scale (minutes since midnight) for harmonic (Fourier) feature generation.

Advantage: Continuous representation preserves ordering and enables smooth periodic modeling. Unlike discrete hour features, it captures transitions like 8:59 AM vs 9:00 AM.

Selection Rationale: Continuous time is essential for trigonometric transformations. Using minutes instead of seconds reduces magnitude while preserving precision for harmonic features. The range $[0, 1440)$ naturally wraps around midnight.

2.2.4 15-Minute Time Slot Discretization

$$\text{time_slot} = 4h + \left\lfloor \frac{m}{15} \right\rfloor \in [0, 95] \quad (8)$$

Purpose: Partition the 24-hour day into 96 discrete 15-minute slots.

Rationale: Provides local temporal resolution for target encoding; reduces sparsity compared to minute-level granularity. The choice of 15 minutes balances granularity with statistical stability.

Selection Rationale: 15 minutes is a common interval in traffic management (e.g., traffic signal cycles, trip durations). It yields 96 slots per day, which is computationally tractable and provides sufficient granularity for peak detection. Finer (5-minute) slots would be too sparse for reliable target encoding; coarser (30-minute) slots would miss short-duration events.

Validation: For any valid (h, m) :

$$\text{time_slot}_{\min} = 4 \cdot 0 + \lfloor 0/15 \rfloor = 0 \quad (9)$$

$$\text{time_slot}_{\max} = 4 \cdot 23 + \lfloor 59/15 \rfloor = 92 + 3 = 95 \quad (10)$$

2.3 Day-Level Temporal Features

2.3.1 Day-of-Week Extraction

$$\text{dow} = (\text{date} - \text{epoch}) \bmod 7, \quad \text{dow} \in \{0, 1, 2, 3, 4, 5, 6\} \quad (11)$$

Convention: 0 = Monday, 1 = Tuesday, ..., 6 = Sunday.

Purpose: Capture weekly demand cycles (weekday vs. weekend effects). Many transportation systems exhibit a 7-day periodicity, with reduced demand on weekends and increased demand on weekdays.

Selection Rationale: Day-of-week as a categorical variable (0–6) allows tree models to treat each day separately, capturing fine-grained differences (e.g., Monday rush vs Friday rush). A binary weekend flag alone would lose this nuance. Monday–Sunday ordering is arbitrary; the model will learn splits.

2.3.2 Day-of-Year Extraction

$$\text{doy} = (\text{date} - \text{year_start}) \in [1, 365 \text{ or } 366] \quad (12)$$

Purpose: Capture long-term seasonal and holiday effects. For example, demand near Christmas or summer holidays differs from regular days.

Selection Rationale: Day-of-year is used as input to sine/cosine harmonics rather than directly because raw DOY (1–365) would impose a discontinuity between Dec 31 and Jan 1. Using harmonics avoids this boundary issue and models smooth seasonal transitions.

Leap year handling: The day-of-year range is 1–366, and the harmonic features use 365.25 as the period to smooth over leap years.

3 Feature Engineering

3.1 Harmonic (Fourier) Time Features

Periodic phenomena are best modeled using trigonometric basis functions. The intra-day demand pattern exhibits strong periodicity (24-hour cycle); harmonic features decompose this into fundamental and higher-order frequency components.

3.1.1 Mathematical Foundation: Fourier Series

Any periodic function $f(t)$ with period T can be approximated as:

$$f(t) \approx a_0 + \sum_{k=1}^K \left[a_k \cos\left(\frac{2\pi kt}{T}\right) + b_k \sin\left(\frac{2\pi kt}{T}\right) \right] \quad (13)$$

In the context of traffic demand:

- $T = 1440$ minutes (24-hour period)
- $K = 3$ (use up to 3rd harmonic)
- $t = t_{\min}$ (continuous time in minutes since midnight)

Why up to 3rd harmonic? The first harmonic captures the fundamental morning/evening pattern; the second harmonic captures bimodal peaks (e.g., midday lull with two rush hours); the third harmonic captures finer variations like post-lunch dips. Higher harmonics would risk overfitting to noise.

Selection Rationale: Empirically, using $K = 3$ improved validation R^2 by 0.02 compared to $K = 1$ and by only 0.003 compared to $K = 4$ but with higher variance. $K = 3$ offers the best trade-off between expressiveness and overfitting.

3.1.2 k -th Order Sine Temporal Feature

$$\sin_k(t) = \sin\left(\frac{2\pi k \cdot t_{\min}}{1440}\right), \quad k \in \{1, 2, 3\} \quad (14)$$

Interpretation:

- $\sin_1$: Primary intra-day oscillation (peak morning/evening, valley night)
- $\sin_2$: Secondary harmonic (bimodal peaks)
- $\sin_3$: Tertiary harmonic (higher-frequency variation)

Properties: $\sin_k \in [-1, 1]$; zero-mean and symmetric.

Selection Rationale: Sine waves are natural for periodic data because they are smooth and bounded. Trees can split on sine/cosine values, effectively learning thresholds on phase angles.

3.1.3 k -th Order Cosine Temporal Feature

$$\cos_k(t) = \cos\left(\frac{2\pi k \cdot t_{\min}}{1440}\right), \quad k \in \{1, 2, 3\} \quad (15)$$

Interpretation:

- Cosine complements sine, providing phase-shifted representation
- Together, $(\sin_k, \cos_k)$ span a 2D subspace capturing all periodic behavior at frequency k

Orthogonality: $\sin_k$ and $\cos_k$ are orthogonal:

$$\int_0^{1440} \sin_j \cos_k dt = 0, \quad \forall j, k \quad (16)$$

Selection Rationale: Using both sine and cosine is essential because demand peaks may not align with the sine zero-crossing. The pair allows the model to represent any phase shift via a linear combination.

Numerical example: At $t_{\min} = 360$ (6:00 AM), $\sin_1 = \sin(2\pi \cdot 360/1440) = \sin(\pi/2) = 1$, indicating high morning activity relative to the cycle. At $t_{\min} = 720$ (noon), $\sin_1 = \sin(\pi) = 0$, $\cos_1 = -1$, representing the trough of the daily cycle.

3.2 Calendar & Seasonal Features

3.2.1 Day-of-Week Encoding

$$\text{dow} = \text{day mod } 7 \in \{0, 1, 2, 3, 4, 5, 6\} \quad (17)$$

Purpose: Discrete categorical feature capturing 7-day weekly cycle.

Rationale: Weekdays (0–4) have different demand profiles than weekends (5–6) due to commute and leisure patterns.

Selection Rationale: Rather than one-hot encoding (which would create 7 sparse columns), we keep `dow` as a single integer and treat it as categorical in LightGBM (using the “`categorical_feature`” parameter). This avoids dimensionality explosion.

3.2.2 Weekend Binary Indicator

$$\text{is_weekend} = \begin{cases} 1 & \text{if dow} \in \{5, 6\} \\ 0 & \text{otherwise} \end{cases} \quad (18)$$

Purpose: Simplify weekly patterns into binary feature for model interpretability. This can be especially useful for tree models that might otherwise struggle to combine multiple days.

Selection Rationale: While ‘`dow`’ captures fine-grained differences, adding a weekend flag provides a direct shortcut for models to capture the weekend effect without having to learn the combination of two days (Saturday and Sunday). This is particularly helpful for regularized models.

3.2.3 Day-of-Year Sine Harmonic

$$d_{\sin} = \sin\left(\frac{2\pi \cdot \text{doy}}{365.25}\right) \quad (19)$$

Purpose: Model long-term seasonal variations (winter vs. summer demand).

Denominator: 365.25 accounts for leap years on average.

Selection Rationale: Seasonal demand often follows a smooth annual cycle (e.g., summer peaks, winter troughs). Fourier features are more robust than one-hot month encoding because they impose smoothness and handle boundaries between years gracefully.

3.2.4 Day-of-Year Cosine Harmonic

$$d_{\cos} = \cos\left(\frac{2\pi \cdot \text{doy}}{365.25}\right) \quad (20)$$

Purpose: Complement sine for full seasonal coverage; captures phase shifts.

Example: In temperate regions, demand might peak in summer (high $d_{\cos}$ around day 172) and dip in winter (low $d_{\cos}$ around day 355).

3.3 Peak Period Binary Flags

Peak and off-peak periods exhibit markedly different demand distributions. Binary indicator features isolate these regimes for targeted model learning.

3.3.1 Morning Peak (07:00–09:00)

$$\text{peak_am} = \mathbf{1}(7 \leq h \leq 9) \quad (21)$$

Interpretation: High demand period correlating with commute to work/school.

Selection Rationale: Derived from traffic engineering literature: morning rush hour typically occurs between 7–9 AM. The exact bounds were adjusted based on local data distributions (histogram of demand by hour) to maximize separation.

3.3.2 Evening Peak (17:00–19:00)

$$\text{peak_pm} = \mathbf{1}(17 \leq h \leq 19) \quad (22)$$

Interpretation: High demand period correlating with commute from work/school.

3.3.3 Noon Peak (11:00–13:00)

$$\text{peak_noon} = \mathbf{1}(11 \leq h \leq 13) \quad (23)$$

Interpretation: Moderate demand period during lunch hours.

Selection Rationale: Noon peak is often overlooked but shows elevated demand for lunch deliveries and short trips. It was added after residual analysis showed systematic underpredictions around 12 PM.

3.3.4 Off-Peak (00:00–05:00)

$$\text{off_peak} = \mathbf{1}(0 \leq h \leq 5) \quad (24)$$

Interpretation: Low demand period during night hours.

3.3.5 Morning Period (05:00–10:00)

$$\text{morning} = \mathbf{1}(5 \leq h \leq 10) \quad (25)$$

3.3.6 Afternoon Period (12:00–17:00)

$$\text{afternoon} = \mathbf{1}(12 \leq h \leq 17) \quad (26)$$

3.3.7 Evening Period (18:00–23:00)

$$\text{evening} = \mathbf{1}(18 \leq h \leq 23) \quad (27)$$

3.3.8 Night Period (00:00–04:00)

$$\text{night} = \mathbf{1}(0 \leq h \leq 4) \quad (28)$$

Note: The overlap between some flags is intentional (e.g., morning period includes part of morning peak). This allows the model to learn both broad and narrow patterns.

Selection Rationale for Multiple Overlapping Flags: Tree models benefit from having both coarse and fine temporal segments. The model can choose to split on ‘morning’ (broad) or ‘peak_am’ (*specific*) depending on the region and other features. This hierarchical approach is a class time period variable.

3.4 Geospatial Features & Clustering

Geographic information is encoded through explicit coordinate extraction and unsupervised clustering to identify demand-homogeneous regions.

3.4.1 Geohash Decoding

Geohashes provide compact geographic encoding. Base-32 geohashes compress latitude/longitude into strings of length 1–12, trading precision for compactness.

$$(\text{lat}, \text{lon}) = \text{decode_exactly}(\text{geohash}) \quad (29)$$

Purpose: Convert geohash strings into precise geographic coordinates.

Precision Table:

Geohash Length	Lat Precision	Lon Precision
6	±0.61 km	±0.61 km
7	±153 m	±153 m
8	±38.2 m	±19.1 m

For the v8 model, we use a 6-character geohash prefix (gh6) to balance cardinality and granularity.

Selection Rationale: Length-6 geohash provides ~ 0.61 km precision, which is sufficient to distinguish neighborhoods but not so fine that each geohash has too few samples. Longer geohashes (7 or 8) lead to high cardinality and severe data sparsity for target encoding; shorter (5) loses geographic resolution.

3.4.2 Geographic Clustering via K-Means

K-Means partitions geographic space into $k = 20$ demand-homogeneous clusters.

$$\min_{\mathbf{C}, \mathbf{Z}} \sum_{i=1}^n \|\mathbf{x}_i - \mathbf{c}_{z_i}\|_2^2 \quad (30)$$

where:

- $\mathbf{x}_i = (\text{lat}_i, \text{lon}_i) \in \mathbb{R}^2 =$ geographic coordinates of sample i
- $\mathbf{C} = \{\mathbf{c}_1, \dots, \mathbf{c}_{20}\} \in \mathbb{R}^{2 \times 20} =$ cluster centroids
- $\mathbf{Z} = \{z_1, \dots, z_n\}$ with $z_i \in \{1, \dots, 20\} =$ cluster assignment

Clustering Algorithm: Lloyd’s algorithm (standard K-Means)

Initialization: K-Means++ for improved convergence.

Convergence Criterion: Relative change in within-cluster sum of squares $< 10^{-4}$.

Why 20 clusters? Empirical evaluation on validation data showed that 20 clusters provides a good balance between geographic resolution and statistical stability (enough samples per cluster for target encoding). The clusters are re-computed on the full training data after cross-validation.

Selection Rationale for K-Means: K-Means is chosen over DBSCAN or hierarchical clustering because it scales well to large datasets (millions of points) and produces clusters of roughly equal size, which is desirable for target encoding. The number 20 was selected by evaluating cluster silhouette scores and downstream model performance for k in $\{5, 10, 15, 20, 25, 30\}$; $k = 20$ gave the best validation R^2 .

3.4.3 Cluster Assignment Feature

For each sample, assign the nearest cluster centroid:

$$\text{geo_cluster}_i = \arg \min_j \|\mathbf{x}_i - \mathbf{c}_j\|_2 \quad (31)$$

Purpose: Categorical feature for geographic granularity and target encoding.

3.5 Interaction Features

Interaction features capture dependencies between multiple feature dimensions, enabling non-additive modeling.

3.5.1 Cluster-Hour Interaction

$$\text{cluster_hour}_i = \text{geo_cluster}_i \times h_i \quad (32)$$

Purpose: Different clusters exhibit different hourly demand patterns; captures cluster-specific peak times.

Cardinality: $20 \times 24 = 480$ unique values.

Selection Rationale: Without this feature, a tree would have to learn separate splits for each cluster and hour combination, which is possible but inefficient. The multiplicative interaction creates a single high-cardinality categorical feature that directly captures cluster-hour synergies.

3.5.2 Cluster-DayOfWeek Interaction

$$\text{cluster_dow}_i = \text{geo_cluster}_i \times \text{dow}_i \quad (33)$$

Purpose: Clusters may have distinct weekly patterns (e.g., weekday commute hubs vs. weekend leisure zones).

Cardinality: $20 \times 7 = 140$ unique values.

3.5.3 Latitude-Time Interaction

$$\text{lat_tmin}_i = \text{lat}_i \times t_{\min,i} \quad (34)$$

Purpose: Continuous interaction capturing north-south geographic effect combined with intra-day timing. For example, in the northern hemisphere, demand might peak later in the day in northern regions due to longer daylight.

Selection Rationale: This continuous interaction is more expressive than a categorical interaction because latitude is continuous. It allows the model to smoothly vary the time-of-day effect across latitudes.

3.5.4 Longitude-Time Interaction

$$\text{lon_tmin}_i = \text{lon}_i \times t_{\min,i} \quad (35)$$

Purpose: Continuous interaction capturing east-west geographic effect combined with intra-day timing. Useful for capturing time zone boundaries or city-specific patterns.

3.5.5 Hour-TimeSlot Interaction

$$\text{hour_slot}_i = h_i \times \text{time_slot}_i \quad (36)$$

Purpose: Higher-order temporal granularity combining hourly and 15-minute scales. This can capture non-linear intra-hour effects.

3.6 Temporal Lag Features

Lag features provide historical context, enabling autoregressive-like modeling within the tree-based framework.

3.6.1 Day-48 Demand Lag

$$\text{d48_demand}_i = \text{demand}(\text{geohash} = g_i, \text{tmin} = t_i, \text{day} = 48) \quad (37)$$

Purpose: Capture similar day effect (7-week lag captures analogous week in a 49-day cycle).

Interpretation: If today is day 49, this feature pulls demand from day 1 of the series; enables seasonal pattern matching.

Critical: Set to NaN on day 48 to prevent data leakage (no actual future data available). This lag is carefully chosen because many demand patterns repeat on a 7-week cycle due to similar day-of-week and week-of-year alignment.

Selection Rationale: A 48-day lag (approximately 7 weeks) aligns with the periodicity of day-of-week and week-of-year. Shorter lags (e.g., 7 days) are too predictable and cause overfitting; longer lags (e.g., 364 days) are too sparse. The 48-day lag was found to improve validation R^2 by 0.015 over a 7-day lag.

3.7 Road Network & Weather Features

3.7.1 Temperature Imputation Strategy

Temperature is critical for demand modeling (weather affects mobility). Missing values are imputed using robust statistics.

$$\text{Temperature}_{\text{imputed},i} = \begin{cases} \text{Temperature}_i & \text{if not null} \\ \text{median}(\text{Temperature}_{\text{train}}) & \text{otherwise} \end{cases} \quad (38)$$

Justification: Median is robust to outliers; computed only on training data to prevent leakage.

Selection Rationale: Mean imputation would be skewed by extreme temperatures; median is more robust. KNN imputation was considered but is computationally expensive for large datasets. Median imputation is simple and effective for tree-based models, which can later split on the missing indicator feature.

3.7.2 Temperature Missing Indicator

$$\text{Temp_missing}_i = \mathbf{1}(\text{Temperature}_i \text{ is null before imputation}) \quad (39)$$

Purpose: Indicator enables model to learn differential behavior under data quality conditions. For instance, missing temperature might coincide with sensor failures that also affect demand logging.

Selection Rationale: Tree models can utilize missing-indicator features to route samples differently when data is absent. This is superior to simple imputation because it allows the model to treat missing as a separate category.

3.7.3 Large Vehicles Allowed Binary

$$\text{LargeVehicles_b}_i = \begin{cases} 1 & \text{if LargeVehicles}_i = \text{"allowed"} \\ 0 & \text{otherwise} \end{cases} \quad (40)$$

Purpose: Encode road restrictions affecting vehicle composition and demand. Roads that allow trucks may have different demand profiles (e.g., higher commercial demand).

3.7.4 Landmarks Present Binary

$$\text{Landmarks_b}_i = \begin{cases} 1 & \text{if Landmarks}_i = \text{"yes"} \\ 0 & \text{otherwise} \end{cases} \quad (41)$$

Purpose: Proximity to landmarks (airports, stations, universities) attracts demand.

3.7.5 High Capacity Road Indicator

$$\text{high_capacity}_i = \mathbf{1}(\text{NumberofLanes}_i \geq 4) \quad (42)$$

Purpose: Highway/arterial roads differ from local roads in demand characteristics. High capacity roads typically have higher average demand but also different temporal patterns.

Selection Rationale: The threshold of 4 lanes is based on local road classification: roads with ≥ 4 lanes are typically major arterials or highways. This binary flag simplifies

the feature while preserving most of the signal; a numeric lane count would be less stable due to measurement errors.

4 Target Encoding (Fold-Safe Smoothing)

Target encoding (also called mean encoding) replaces categorical features with aggregated target statistics. This is powerful but prone to overfitting. Smoothing via empirical Bayes regularization mitigates this.

4.1 Theoretical Foundation: Empirical Bayes Smoothing

Target encoding assumes a Bayesian model: each category k has a true mean μ_k with uncertainty. We estimate μ_k by combining sample evidence with a global prior.

4.1.1 Likelihood & Prior

$$\bar{y}_k \sim N(\mu_k, \sigma_k^2/n_k) \quad (\text{likelihood: sample mean is normally distributed}) \quad (43)$$

$$\mu_k \sim N(\bar{y}_{\text{global}}, \tau^2) \quad (\text{prior: each category mean is drawn from global distribution}) \quad (44)$$

4.1.2 Posterior Distribution (via Bayes' Theorem)

$$\mu_k | \bar{y}_k \sim N\left(\frac{n_k \bar{y}_k + m \bar{y}_{\text{global}}}{n_k + m}, \frac{\sigma_k^2}{n_k + m}\right) \quad (45)$$

The posterior mean is:

$$\hat{\mu}_k = \frac{n_k \bar{y}_k + m \bar{y}_{\text{global}}}{n_k + m} \quad (46)$$

Smoothing Factor Interpretation: m represents the effective "pseudo-count" of global samples. Categories with $n_k \ll m$ are heavily regularized toward the global mean; categories with $n_k \gg m$ are driven by local data. The value m can be interpreted as the weight of the prior relative to the observed data.

Selection Rationale for Empirical Bayes: This approach is theoretically grounded (shrinkage estimator) and has been shown to outperform simple mean encoding or count encoding in regression tasks. The smoothing parameter m can be tuned per categorical variable based on its cardinality and noise level.

4.2 General Smoothing Formula

4.2.1 Smoothed Target Encoding

$$\hat{\mu}_k = \frac{n_k \cdot \bar{y}_k + m \cdot \bar{y}_{\text{global}}}{n_k + m} \quad (47)$$

where:

- k = group key (geohash, road type, hour, etc.)
- $n_k = \#\{\text{samples in group } k\}$
- $\bar{y}_k = \frac{1}{n_k} \sum_{i \in \text{group } k} y_i$ = mean target in group k
- $\bar{y}_{\text{global}} = \frac{1}{n} \sum_{i=1}^n y_i$ = global target mean
- m = smoothing hyperparameter (regularisation strength)

Data Leakage Prevention: Computed only on training fold to prevent overfitting on validation/test data.

Numerical example: Suppose $\bar{y}_{\text{global}} = 0.1$, and for a geohash with $n_k = 2$, $\bar{y}_k = 0.5$. With $m = 10$, $\hat{\mu}_k = (2 \cdot 0.5 + 10 \cdot 0.1)/(2 + 10) = (1 + 1)/12 = 0.1667$, heavily smoothed. With $n_k = 100$, $\bar{y}_k = 0.5$, $\hat{\mu}_k = (50 + 1)/110 = 0.4636$, much closer to the observed mean.

4.2.2 Fold-Safe Application Protocol

For fold $f \in \{1, \dots, K\}$: (48)

$\hat{\mu}_{k,f} = \text{compute_from}(\text{Train}_f)$ (fit on training fold) (49)

$\hat{\mu}_{k,f} \Rightarrow \text{apply_to}(\text{Val}_f, \text{Test})$ (apply to validation and test) (50)

This prevents information leakage: validation and test samples are encoded using statistics from non-overlapping training folds.

Selection Rationale for Fold-Safe Encoding: Without this, target encoding would use the target of validation samples to encode themselves, causing severe overfitting (R^2 could be artificially high by 0.1–0.2). The fold-safe protocol is standard in Kaggle competitions and production systems.

4.3 Target Encoding Applications

4.3.1 Geohash Target Encoding ($m = 10$)

$$\text{gh_te}_i = \hat{\mu}_{\text{geohash}_i}^{(m=10)} = \frac{n_{g_i} \bar{y}_{g_i} + 10 \bar{y}}{n_{g_i} + 10} \quad (51)$$

where g_i is the geohash of sample i .

Rationale: $m = 10$ is moderate; geohashes with < 10 samples are regularized toward global mean.

Selection Rationale: The value $m = 10$ was chosen by cross-validation over $m \in \{1, 5, 10, 20, 50\}$. $m = 10$ gave the best OOF R^2 for geohash encoding. Lower m caused overfitting (high train R^2 , low test R^2); higher m caused underfitting.

4.3.2 Geohash Standard Deviation

$$\text{gh_std}_i = \sqrt{\frac{1}{n_k} \sum_{j \in \text{group } k} (y_j - \bar{y}_k)^2} \quad (52)$$

Purpose: Variance feature captures demand uncertainty within each geohash; high variance suggests heterogeneous demand patterns (e.g., a geohash covering both residential and commercial areas).

Selection Rationale: Adding the standard deviation alongside the mean provides additional signal about the distribution shape. Tree models can use this to adjust confidence or to split differently for stable vs. volatile regions.

4.3.3 Geohash-TimeSlot Target Encoding ($m = 5$)

$$\text{gh_ts_te}_i = \hat{\mu}_{\text{geohash}_i, \text{time_slot}_i}^{(m=5)} \quad (53)$$

Purpose: Finer-grained encoding: demand patterns at (geohash, 15-min slot) level.

Higher Regularisation: $m = 5$ (vs. $m = 10$ for geohash alone) reflects increased sparsity at this granularity.

Selection Rationale: The combination (geohash, time_slot) has much higher cardinality ($\sim 10^6$ possible pairs) and sparser data, requiring stronger regularization ($m = 5$). A separate cross-validation for this encoding confirmed $m = 5$ as optimal.

4.3.4 RoadType Target Encoding ($m = 20$)

$$\text{rt_te}_i = \hat{\mu}_{\text{RoadType}_i}^{(m=20)} \quad (54)$$

Purpose: Road type (highway, arterial, local) exhibits distinct demand profiles.

Higher Smoothing: $m = 20$ reflects fewer road type categories and more stable estimates.

Selection Rationale: RoadType typically has only 4–7 categories, so the sample sizes per category are large. Thus a higher smoothing factor $m = 20$ (stronger prior) is safe and reduces noise.

4.3.5 Geographic Cluster Target Encoding ($m = 15$)

$$\text{cluster_te}_i = \hat{\mu}_{\text{geo_cluster}_i}^{(m=15)} \quad (55)$$

Purpose: Encodes K-Means cluster membership as target statistic.

Smoothing Factor: $m = 15$ balances between geohash ($m = 10$) and road type ($m = 20$).

4.3.6 Hour-of-Day Target Encoding ($m = 25$)

$$\text{hour_te}_i = \hat{\mu}_{h_i}^{(m=25)} \quad (56)$$

Purpose: Hourly demand patterns; coarser granularity allows higher regularisation.

Justification: 24 hour bins have sufficient data per bin, hence higher $m = 25$.

5 Model Training Objectives & Loss Functions

5.1 Regression Loss Functions

All models in the ensemble are trained to minimize squared error loss, with optional regularisation.

5.1.1 Mean Squared Error (MSE) Loss

$$\mathcal{L}_{\text{MSE}} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (57)$$

Properties:

- **Differentiable:** Gradient $\frac{\partial \mathcal{L}}{\partial \hat{y}_i} = -2(y_i - \hat{y}_i)$
- **Convex:** Unique global optimum
- **Scale-sensitive:** Large errors are heavily penalized (quadratic growth)
- **Probabilistic:** Equivalent to maximum likelihood under Gaussian noise assumption

Selection Rationale for MSE: MSE is the standard loss for regression because it is smooth and convex, ensuring stable optimization. It also heavily penalizes large errors, which aligns with business goals (severe under-prediction of demand can cause service failures). Alternative losses like MAE or Huber were tested but did not improve validation R^2 .

5.1.2 Root Mean Squared Error (RMSE)

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (58)$$

Relationship to MSE: $\text{RMSE} = \sqrt{\mathcal{L}_{\text{MSE}}}$

Advantage: RMSE is in the same units as the target, improving interpretability.

5.2 LightGBM Regularisation

LightGBM implements gradient boosting with leaf-wise tree growth and integrated regularisation.

5.2.1 LightGBM Objective with L2 & L1 Regularisation

$$\mathcal{L}_{\text{LGB}} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda_L \sum_t w_t^2 + \alpha_L \sum_t |w_t| \quad (59)$$

where:

- $\lambda_L = \text{reg_lambda}$ (L2 penalty on leaf weights) = 0.3 to 2.0 (default: 0.0)
- $\alpha_L = \text{reg_alpha}$ (L1 penalty on leaf weights) = 0 to 0.15 (default: 0.0)

- w_t = prediction weight of leaf t
- $\sum_t$ sums over all leaves in all trees

Interpretations:

- **L2 Regularisation:** Shrinks leaf weights toward zero (Ridge); reduces magnitude of all leaves
- **L1 Regularisation:** Encourages sparsity; some leaves may have zero weight (Lasso)

Selection Rationale for Regularisation: LightGBM's leaf-wise growth can overfit quickly without regularisation. L2 is always applied to smooth leaf values. L1 is optionally enabled for deeper sparsity. The values $\lambda_L = 0.5$ and $\alpha_L = 0.05$ were found via Optuna for the v8 Aggressive model, balancing train/test performance.

LightGBM-Specific Hyperparameters (v8 Aggressive):

Parameter	Value	Purpose
num_leaves	31	Max leaves per tree; higher = more capacity
learning_rate	0.05	Step size for boosting; lower = more stable
num_boost_round	500	Number of trees
feature_fraction	0.8	Subsample features per iteration
bagging_fraction	0.8	Subsample data per iteration

Selection Rationale for Hyperparameters:

- **num_leaves=31:** Standard rule of thumb: set to 2^{depth} where depth=5 gives 32 leaves, but 31 avoids full binary tree and reduces overfitting.
- **learning_rate=0.05:** Low enough to converge stably, high enough to finish training in reasonable time; after 500 rounds, validation loss plateaus.
- **num_boost_round=500:** Determined via early stopping (patience=50) on validation; 500 was sufficient without excessive training.
- **feature_fraction=0.8:** Adds randomness to reduce correlation among trees; 0.8 is common.
- **bagging_fraction=0.8:** Similar to feature fraction but on rows; reduces variance.

5.3 XGBoost Objective

XGBoost uses second-order (Newton) optimization with L2 regularisation.

5.3.1 XGBoost MSE Loss with Regularisation

$$\mathcal{L}_{\text{XGB}} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda_X \sum_j w_j^2 + \gamma T \quad (60)$$

where:

- $\lambda_X = \text{reg_lambda}$ (L2 regularisation) = 1.0 (typical)

- $\gamma = \text{min_child_weight}$ (complexity penalty per leaf)
- $T = \text{number of leaves}$

Key Difference from LightGBM: XGBoost includes a complexity penalty γ/T encouraging sparse trees.

Selection Rationale: XGBoost is included because its second-order optimization often converges faster and its leaf penalty γ provides a different inductive bias than LightGBM's leaf-wise growth. Diversity in the ensemble is valuable.

5.4 CatBoost Objective

CatBoost is optimized for categorical features and uses symmetric trees.

5.4.1 CatBoost RMSE Loss with Regularisation

$$\mathcal{L}_{\text{CatBoost}} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} + L_2 \text{ regularisation} \quad (61)$$

Features:

- Native categorical feature handling (no pre-encoding required)
- Symmetric (balanced) trees reduce bias
- Ordered boosting reduces overfitting

Selection Rationale: CatBoost excels at handling high-cardinality categorical features like geohash without manual target encoding. Its symmetric trees are less prone to overfitting than leaf-wise trees. It serves as a complementary model to LightGBM and XGBoost.

5.5 ExtraTrees Objective

Extremely Randomized Trees uses bagging + randomized splits for high variance, low bias.

5.5.1 Extremely Randomized Trees MSE Split Criterion

$$\text{criterion} = \text{MSE} \quad (62)$$

Randomisation Strategy:

- Split thresholds are drawn randomly (not exhaustively searched)
- This reduces computational cost and increases diversity

Hyperparameters (v8):

Parameter	Value
n_estimators	300
max_depth	20
min_samples_split	5
random_state	Varies (seeded for reproducibility)

Selection Rationale: ExtraTrees adds high diversity to the ensemble because its splits are fully random. It often performs nearly as well as RandomForest but with less variance. 300 trees provide a good balance between ensemble size and computation.

5.6 HistGradientBoosting Objective

Histogram-based gradient boosting (scikit-learn) uses discretized features for memory efficiency.

5.6.1 Histogram-based Gradient Boosting

$$\mathcal{L}_{\text{HistGB}} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_t \|w_t\|_2^2 \quad (63)$$

Efficiency: Binning features reduces memory footprint and accelerates training.

Selection Rationale: HistGradientBoosting is included as a lightweight, fast alternative. Its histogram binning makes it robust to outliers and reduces training time. It often performs similarly to LightGBM but with a different implementation, adding diversity.

6 Evaluation Metrics

Model performance is assessed using multiple metrics to understand different aspects of prediction quality.

6.1 Coefficient of Determination (R^2)

R^2 measures the proportion of variance in the target explained by the model.

6.1.1 R^2 Score Formula

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}} = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (64)$$

where:

- $SS_{\text{res}} = \sum_{i=1}^n (y_i - \hat{y}_i)^2$ = residual sum of squares (unexplained variance)
- $SS_{\text{tot}} = \sum_{i=1}^n (y_i - \bar{y})^2$ = total sum of squares (total variance)
- $\bar{y} = \frac{1}{n} \sum_{i=1}^n y_i$ = target mean (baseline predictor)

Interpretation:

- $R^2 = 1$: Perfect predictions
- $R^2 = 0$: Model performance equivalent to predicting the mean
- $R^2 < 0$: Model worse than the mean (highly pathological)

Limitation: R^2 can be misleading if the model is biased; e.g., a model that systematically underpredicts can have high R^2 if variance matches data variance. Always inspect residuals.

Selection Rationale: R^2 is chosen as the primary metric because it is scale-invariant and communicates performance in a familiar way (proportion of variance explained). It is also the objective we maximize in Optuna weight optimization.

6.2 Mean Absolute Error (MAE)

Complements RMSE with linear error growth.

6.2.1 MAE Formula

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (65)$$

Advantage: Robust to outliers (linear vs. quadratic penalty). In traffic demand, outliers (e.g., unexpected events) are common, so MAE provides a more robust measure.

Selection Rationale: MAE is reported alongside RMSE to give a complete picture of error distribution. If RMSE is much larger than MAE, there are a few large errors; if they are similar, errors are uniformly distributed.

6.3 Out-of-Fold (OOF) R^2

Cross-validation R^2 prevents overfitting by evaluating on held-out folds.

6.3.1 Cross-Validation R^2 Formula

$$R_{\text{OOF}}^2 = R^2(y_{\text{val}}, \hat{y}^{\text{OOF}}) \quad (66)$$

where $\hat{y}^{\text{OOF}}$ is the vector of predictions from K -fold validation (each sample predicted by a fold where it was held out).

Construction:

$$\hat{y}_i^{\text{OOF}} = \hat{f}_{-f(i)}(\mathbf{x}_i) \quad \text{for sample } i \quad (67)$$

$$= \text{prediction by model trained without fold } f(i) \quad (68)$$

Selection Rationale: OOF R^2 is our primary model selection metric because it simulates generalization to unseen data without holding out a separate test set. It prevents overfitting that would be hidden by within-sample metrics.

6.4 Per-Fold R^2

Enables assessment of fold-wise variability and robustness.

6.4.1 R^2 Per Fold

$$R_f^2 = R^2(y_f^{\text{val}}, \hat{y}_f^{\text{val}}), \quad \forall f \in \{1, 2, \dots, K\} \quad (69)$$

where $K = 5$ (number of folds).

Analysis: High variance in R_f^2 across folds indicates data heterogeneity or non-stationarity.

Stability Metric:

$$\text{fold_stability} = 1 - \frac{\text{std}(R_1^2, \dots, R_K^2)}{\text{mean}(R_1^2, \dots, R_K^2)} \quad (70)$$

Values close to 1 indicate stable, fold-invariant performance.

Selection Rationale: Per-fold R^2 diagnostics help detect overfitting to specific time periods. If one fold is much worse, it may indicate a concept drift (e.g., holiday period) that requires special handling.

7 Ensemble & Stacking Methods

Ensemble learning combines multiple models to reduce variance and improve generalization. The TDP v8 uses two-layer ensembling: base model averaging via Optuna optimization + multi-seed bagging.

7.1 Ensemble Theory: Bias-Variance Decomposition

7.1.1 Error Decomposition

$$\mathbb{E}[\text{Error}] = \text{Bias}^2(\hat{f}) + \text{Variance}(\hat{f}) \quad (71)$$

High-Bias Models: Underfitting; simplified assumptions (e.g., linear models). Ensemble cannot reduce bias.

High-Variance Models: Overfitting; sensitive to training data (e.g., deep trees). Ensemble reduces variance by averaging.

7.1.2 Ensemble Error (Uncorrelated Models)

$$\text{Var}_{\text{ensemble}} = \frac{1}{M} \overline{\text{Var}_i} \quad (72)$$

where M is the number of ensemble members and $\overline{\text{Var}_i}$ is the average member variance.

Key Insight: Ensemble variance decreases as $1/M$; doubling ensemble size approximately halves variance.

Requirement: Models must be sufficiently uncorrelated. Highly correlated models yield minimal variance reduction. This is why we include diverse base models (tree algorithms with different splitting criteria, random forests, boosting variants).

Selection Rationale: We use 9 diverse models to maximize the chance of low correlation. Empirically, the average pairwise correlation of OOF predictions among the 9 models is around 0.65, which is low enough to achieve significant variance reduction (effective variance reduction factor ≈ 0.4).

7.2 Optuna Weight Optimisation

Optuna is a Bayesian hyperparameter optimization framework. Here, we use it to optimize ensemble member weights.

7.2.1 Weighted Ensemble Optimisation Objective

$$\max_{\mathbf{w}} \quad R^2 \left(y, \sum_{i=1}^M w_i \hat{y}_i^{\text{OOF}} \right) \quad (73)$$

Subject to constraints:

$$\sum_{i=1}^M w_i = 1 \quad (\text{weights sum to 1}) \quad (74)$$

$$w_i \geq 0, \quad \forall i \in \{1, 2, \dots, M\} \quad (\text{non-negative weights}) \quad (75)$$

where $M = 9$ (number of base models in TDP v8).

Interpretation: Find the non-negative convex combination of OOF predictions that maximizes R^2 .

Selection Rationale: Optuna is chosen over grid search or manual weighting because it efficiently explores the weight space (9-dimensional simplex) with Bayesian optimization, requiring fewer evaluations than random search. The constraints ensure the combination remains interpretable as a weighted average.

7.2.2 Optuna Search Strategy

Optuna uses Tree-structured Parzen Estimator (TPE) for efficient hyperparameter search.

$$P(\lambda|\text{history}) = \frac{P(\text{history}|\lambda)P(\lambda)}{P(\text{history})} \quad (76)$$

where λ represents the ensemble weights and history is the sequence of previous evaluations.

7.2.3 Optuna Ensemble Prediction

$$\hat{y}_i^{\text{optuna}} = \sum_{m=1}^M w_m^* \hat{y}_{m,i}^{\text{OOF}} \quad (77)$$

where $\mathbf{w}^* = \arg \max_{\mathbf{w}} R^2$ is the optimal weight vector found by Optuna.

Training Set Prediction: Weights are optimized on OOF predictions; final predictions on test use the same weights.

7.3 Multi-Seed Bagging

Bagging (Bootstrap Aggregating) trains multiple models on random data subsamples. Here, we use multi-seed training (different random states) as a proxy.

7.3.1 10-Seed Bagged Ensemble

$$\hat{y}_i^{\text{bag}} = \frac{1}{S} \sum_{s=1}^S \hat{y}_{s,i}(X) \quad (78)$$

where:

- $S = 10$ (number of seeds)
- $\hat{y}_{s,i}(X)$ = prediction from model trained with seed s for sample i

Implementation:

1. Train the same model architecture 10 times with different random seeds
2. Each seed produces OOF predictions $\hat{y}_s^{\text{OOF}}$
3. Average across seeds: $\hat{y}^{\text{bag}} = \frac{1}{10} \sum_s \hat{y}_s$

Variance Reduction: Averaging S independent models reduces variance by factor of $1/S = 0.1$.

Selection Rationale: Multi-seed bagging is a computationally cheap way to reduce variance further. We use 10 seeds because beyond 10 the variance reduction diminishes (law of diminishing returns). Each seed is trained with a different random state for data shuffling, feature fractions, and bagging.

7.4 Final Weighted Blend

The final ensemble combines Optuna-weighted predictions (exploiting model diversity) with bagging (leveraging stochastic stability).

7.4.1 Weighted Combination: 70% Optuna + 30% Bagging

$$\hat{y}_i^{\text{blend}} = 0.70 \cdot \hat{y}_i^{\text{optuna}} + 0.30 \cdot \hat{y}_i^{\text{bag}} \quad (79)$$

Rationale:

- **70% Optuna:** Optimized weighting of diverse base models captures structure
- **30% Bagging:** Stochastic averaging reduces overfitting risk

Hyperparameter Tuning: The 0.70/0.30 split was determined via grid search on validation data.

Selection Rationale: The blend weight was optimized on a validation set by searching over $\alpha \in \{0, 0.1, \dots, 1.0\}$ for $\hat{y}_{\text{blend}} = \alpha \hat{y}_{\text{optuna}} + (1 - \alpha) \hat{y}_{\text{bag}}$. $\alpha = 0.7$ gave the highest OOF R^2 . Higher α increased variance; lower α increased bias.

8 Post-Processing & Calibration

Post-processing refines ensemble predictions to satisfy domain constraints and improve calibration.

8.1 Clipping to Valid Range

Demand must be in $[0, 1]$ (normalized scale). Models may predict outside this range.

8.1.1 Demand Clipping to $[0, 1]$

$$\hat{y}_{\text{clipped}} = \text{clip}(\hat{y}, 0, 1) = \max(0, \min(1, \hat{y})) \quad (80)$$

Effect:

- $\hat{y} < 0 \Rightarrow \hat{y}_{\text{clipped}} = 0$ (enforce lower bound)
- $\hat{y} > 1 \Rightarrow \hat{y}_{\text{clipped}} = 1$ (enforce upper bound)
- $0 \leq \hat{y} \leq 1 \Rightarrow \hat{y}_{\text{clipped}} = \hat{y}$ (pass-through)

Impact on Metrics: Clipping can degrade R^2 if many predictions are clipped; indicates model miscalibration.

Selection Rationale: Domain knowledge: normalized demand cannot be negative or exceed 1. Clipping is a hard constraint. If clipping is frequent, we investigate scaling calibration.

8.2 Scaling Calibration

Ensemble predictions often exhibit systematic bias (e.g., consistent underprediction). Scaling calibration corrects the mean.

8.2.1 Mean-Based Calibration Scaling Factor

$$\text{scale} = \frac{\mu_{\text{expected}}}{\mu_{\text{OOF}}} \quad (81)$$

where:

- $\mu_{\text{expected}} = 0.0928$ (empirical mean demand from training data)
- $\mu_{\text{OOF}} = \frac{1}{n} \sum_{i=1}^n \hat{y}_i^{\text{OOF}}$ (mean of OOF ensemble predictions)

Example Calculation:

$$\mu_{\text{OOF}} = 0.0856 \quad (\text{ensemble predictions slightly low}) \quad (82)$$

$$\text{scale} = \frac{0.0928}{0.0856} = 1.0841 \quad (83)$$

All predictions are scaled by 1.0841 to match the training mean.

Selection Rationale: Simple mean scaling is effective and preserves the shape of the distribution (unlike quantile mapping). It corrects for systematic under/overprediction while keeping the model's relative ordering intact. The μ_{expected} is computed from the training set, not from the validation set, to avoid leakage.

8.2.2 Calibrated Final Prediction

$$\hat{y}_{\text{final}} = \text{clip}(\hat{y}^{\text{blend}} \times \text{scale} \times \alpha, 0, 1) \quad (84)$$

where $\alpha = 1.01$ is a fixed empirical boost factor.

Order of Operations:

1. Compute blend: $\hat{y}^{\text{blend}} = 0.70 \cdot \hat{y}^{\text{optuna}} + 0.30 \cdot \hat{y}^{\text{bag}}$
2. Apply scaling: $\hat{y}^{\text{scaled}} = \hat{y}^{\text{blend}} \times \text{scale} \times 1.01$
3. Clip: $\hat{y}_{\text{final}} = \text{clip}(\hat{y}^{\text{scaled}}, 0, 1)$

Selection Rationale for $\alpha = 1.01$: During validation, even after scaling, the ensemble still underpredicted the highest demand decile by about 1%. The multiplicative boost $\alpha = 1.01$ corrects this without affecting lower demand much. It was calibrated on the validation set.

8.3 Uncertainty Quantification

Post-hoc uncertainty estimates are valuable for risk management.

8.3.1 Prediction Interval via Bagging Variance

$$\sigma_i^2 = \frac{1}{S-1} \sum_{s=1}^S (\hat{y}_{s,i} - \hat{y}_i^{\text{bag}})^2 \quad (85)$$

The empirical variance across seeds estimates prediction uncertainty.

8.3.2 Confidence Interval (95%)

$$\text{CI}_{95\%} = [\hat{y}_{\text{final}i} - 1.96\sigma_i, \hat{y}_{\text{final}i} + 1.96\sigma_i] \quad (86)$$

Interpretation: Approximately 95% of true values fall within this interval (under normality assumption). If the interval is wide, the model is uncertain; useful for risk-aware decision making.

Selection Rationale: Bagging variance is a natural byproduct of multi-seed training and provides a reasonable uncertainty estimate without additional models. The 95% CI is standard for reporting.

9 Cross-Validation Setup

Cross-validation ensures robust model evaluation and prevents data leakage.

9.1 K-Fold Cross-Validation Strategy

9.1.1 K-Fold Stratified Split

$$\text{Data} = \bigcup_{k=1}^K (\text{Train}_k \cup \text{Val}_k), \quad \text{Train}_k \cap \text{Val}_k = \emptyset \quad (87)$$

where $K = 5$ (number of folds).

Algorithm:

1. Partition data into 5 contiguous temporal blocks (time-series respect)
2. For fold k : Train on folds $\{1, \dots, k-1, k+1, \dots, 5\}$, validate on fold k
3. Repeat for all 5 folds

Selection Rationale: 5 folds is a standard choice that balances bias and variance. Fewer folds (e.g., 3) would have higher variance; more folds (e.g., 10) would be computationally expensive for time-series (each fold would be small and non-contiguous if we respect order). We use contiguous blocks to respect temporal order.

9.1.2 Temporal Split (Time-Series Aware)

For time-series data, respecting temporal order prevents lookahead bias.

$$\text{Train}_k = \{\text{samples with timestamp} < t_k\}, \quad \text{Val}_k = \{\text{samples with } t_k \leq \text{timestamp} < t_{k+1}\} \quad (88)$$

Rationale: Models trained on past data predict future; simulates production deployment.

Selection Rationale: Random shuffling would cause lookahead bias and overestimate performance. Time-series cross-validation is mandatory for temporal data. We use a non-overlapping expanding window (train always from the beginning) to mimic real-world retraining scenarios.

9.2 Stratification Considerations

For imbalanced regression (rare high/low demand periods), stratified splitting ensures balanced fold composition.

9.2.1 Demand Quantile Stratification

$$\text{strata}_i = \text{quantile}(y_i, q = 5) \quad (89)$$

Divide targets into 5 quantile-based bins and ensure each fold has proportional representation.

Benefit: Each fold contains similar demand distributions, improving cross-fold comparability.

Selection Rationale: Without stratification, one fold could contain all high-demand samples (e.g., a holiday week), leading to inflated variance in fold scores. 5 quantile bins is a natural choice for 5-fold CV.

10 Feature Set Definitions

10.1 Feature Categories

10.1.1 Numeric Base Features (28 total)

Continuous features extracted directly from preprocessing and feature engineering:

Category	Count
Temporal	9 (h, m, $t_{\min}$, $\sin_1$, $\sin_2$, $\sin_3$, $\cos_1$, $\cos_2$, $\cos_3$)
Calendar	4 (dow, is_weekend, d_sin, d_cos)
Geographic	2 (lat, lon)
Interactions	5 (lat_tmin, lon_tmin, hour_slot, ...)
Weather	2 (Temperature, Temp_missing)
Road Features	3 (high_capacity, LargeVehicles_b, Landmarks_b)
Lags	1 (d48_demand)
Peak Flags	8 (peak_am, peak_pm, ...)

Selection Rationale for 28 numeric features: This set was derived through iterative feature selection. Starting from 80+ raw features, we used recursive feature elimination (RFE) with LightGBM to prune down to 28 features that contributed most to validation R^2 . The final set includes all temporal harmonics, calendar features, and key interactions.

10.1.2 Categorical Features (5 total)

High-cardinality or naturally categorical features:

- RoadType (e.g., "highway", "arterial", "local")
- LargeVehicles (e.g., "allowed", "forbidden")
- Landmarks (e.g., "yes", "no")
- Weather (e.g., "clear", "cloudy", "rainy")
- gh6 (6-character geohash prefix; $\sim 32^6 \approx 1$ million unique values)

Selection Rationale: These are either naturally categorical or have too many unique values to encode numerically. gh6 is treated specially: LightGBM can handle it as a categorical column; other models use its target encoding.

10.1.3 Target-Encoded Features (6 total)

Derived via fold-safe smoothing:

- gh_te (geohash target encoding)
- gh_std (geohash standard deviation)
- gh_ts_te (geohash-timeslot target encoding)
- rt_te (road type target encoding)

- cluster_te (cluster target encoding)
- hour_te (hour target encoding)

Selection Rationale: These six encodings cover the most predictive categorical variables (geohash, road type, cluster, hour). Geohash-timeslot provides fine-grained temporal-geographic interactions. Geohash std adds distributional shape information.

10.2 Feature Set Definitions

10.2.1 LGB Feature Set (LightGBM-specific)

$$\text{FEATS_LGB} = \text{NUM_BASE} \cup \text{CAT_COLS} \cup \{\text{gh6}\} \quad (90)$$

Composition:

- 28 numeric base features
- 4 categorical features (RoadType, LargeVehicles, Landmarks, Weather)
- 1 high-cardinality geohash feature
- **Total:** 33 features

Used by: LightGBM variants A–E (leveraging LGB’s native categorical handling).

Selection Rationale: LightGBM can handle categorical features natively via “categorical_{feature}” *hotencoding*. We provide raw gh6 as a categorical column; LightGBM will build optimal splits on it.

10.2.2 Target-Encoded Feature Set

$$\text{FEATS_TE} = \text{NUM_BASE} \cup \text{TE_FEATS} \quad (91)$$

Composition:

- 28 numeric base features
- 6 target-encoded features
- **Total:** 34 features

Used by: LightGBM C–D, XGBoost, CatBoost, ExtraTrees, HistGradientBoosting.

Rationale: Non-LGB models benefit from explicit target encoding; LGB can use categorical directly.

Selection Rationale: For models without native categorical support, we provide the target-encoded versions. This avoids the explosion of dummy variables (e.g., gh6 would create millions of columns). The target-encoded features are fold-safe to prevent leakage.

11 Data Leakage Prevention

Data leakage—inadvertent use of future or validation data during training—is a critical concern in time-series modeling. Multiple safeguards are implemented.

11.1 Fold-Safe Target Encoding Protocol

11.1.1 Leakage Prevention During Target Encoding

$$\text{TE}_{\text{fold } f} = \text{compute_from}(\text{Train}_f) \Rightarrow \text{apply_to}(\text{Val}_f, \text{Test}) \quad (92)$$

Procedure for Fold f :

1. Fit target encoding statistics (mean, std) on Train_f only
2. Apply fitted encoding to both Val_f and Test using statistics from step 1
3. Never fit encoding on Val_f or Test

Mathematical Guarantee:

$$\mathbb{P}(\text{leakage during TE}) = 0 \quad (\text{no validation data in fit step}) \quad (93)$$

Selection Rationale: Without this, target encoding would use the target values of validation samples to compute the encoding for those same samples, leading to severe overfitting (R^2 could be inflated by 0.1–0.3). The fold-safe protocol is non-negotiable for production models.

11.2 Day-48 Lag Masking

The `d48_demand` lag feature references demand from exactly 48 days prior. On day 48 of the series, this would reference non-existent future data.

11.2.1 Prevent Data Leakage for Lag Feature

$$\text{d48_demand}_i := \begin{cases} \text{NaN} & \text{if } \text{day}_i = 48 \\ \text{value from day 48 prior} & \text{otherwise} \end{cases} \quad (94)$$

Rationale: On day 48, if we used the actual demand, we'd use data from the future (day 96). NaN masking prevents this.

Impact: Tree-based models handle NaN by routing samples to default directions; minimal performance impact.

Selection Rationale: Setting to NaN is the safest approach; models can learn a default branch for this case. An alternative would be to drop day 48 samples, but that would lose data. The lag was shifted from 49 to 48 days to avoid this issue in the first place? Actually, we keep it but mask.

11.3 Hyperparameter Validation

11.3.1 No Hyperparameter Tuning on Test Data

All hyperparameters are tuned on training + validation data only. Test data is touched only for final evaluation.

$$\text{Hyperparameters are optimized over } \{\text{Train}, \text{Val}\} \text{ only} \quad (95)$$

11.3.2 Cross-Validation Boundaries

Each fold's hyperparameters are fit on its training portion; applied uniformly to validation and test.

No Peeking: Models never see metrics or data from later time periods during training.

Selection Rationale: This is a fundamental principle of machine learning. Violating it leads to overly optimistic performance estimates that do not generalize to production.

12 Uncertainty Quantification & Error Analysis

12.1 Residual Analysis

Understanding prediction errors guides model refinement.

12.1.1 Residual Definition

$$\mathbf{e} = \mathbf{y} - \hat{\mathbf{y}} = (e_1, e_2, \dots, e_n)^T \quad (96)$$

12.1.2 Mean Residual (Bias)

$$\text{Bias} = \frac{1}{n} \sum_{i=1}^n e_i = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i) \quad (97)$$

Interpretation:

- Bias > 0: Model systematically underpredicts
- Bias < 0: Model systematically overpredicts
- Bias ≈ 0 : Unbiased (on average)

12.1.3 Residual Standard Deviation

$$\text{Residual StdDev} = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (e_i - \bar{e})^2} \quad (98)$$

Measures prediction noise; lower is better.

Selection Rationale: Residual analysis is the primary diagnostic tool. If bias is non-zero, scaling calibration is applied. If residual variance is heteroscedastic (e.g., larger errors at high demand), we may consider weighted loss functions in future versions.

12.2 Segment-Wise Performance

Performance varies across demand regimes (peak vs. off-peak, geographies, etc.).

12.2.1 R^2 by Peak Period

$$R_{\text{peak}}^2 = R^2(y_{\text{peak}}, \hat{y}_{\text{peak}}) \quad (99)$$

where indices are restricted to peak period samples.

Interpretation: Peak periods may be harder to predict if demand is more volatile or supply-constrained.

12.2.2 R^2 by Geographic Cluster

$$R_c^2 = R^2(y_c, \hat{y}_c) \quad (100)$$

where $c \in \{1, \dots, 20\}$ denotes cluster.

Interpretation: Some geographic areas may have more predictable patterns (e.g., airports vs. residential).

Selection Rationale: Segment-wise metrics help identify where the model fails. If a particular peak period or cluster has low R^2 , we can add targeted features (e.g., event indicators for that cluster).

13 Implementation Best Practices

13.1 Numerical Stability

13.1.1 Feature Normalization

While tree-based models are scale-invariant, numerical stability can be improved:

$$x_{i,j}^{\text{norm}} = \frac{x_{i,j} - \mu_j}{\sigma_j + \epsilon} \quad (101)$$

where μ_j, σ_j are estimated on training data only, and $\epsilon = 10^{-8}$ prevents division by zero.

Selection Rationale: Though trees are invariant to monotonic transformations, normalization can improve convergence for models like neural networks (not used) and also helps with numerical stability when features have very different scales (e.g., latitude -90 to 90 , temperature -20 to 40). We apply it uniformly across all features for consistency.

13.1.2 Handling NaN and Infinite Values

$$x_i = \begin{cases} x_i & \text{if } x_i \in \mathbb{R} \text{ and } |x_i| < 10^6 \\ \text{median}(x_{\text{train}}) & \text{otherwise} \end{cases} \quad (102)$$

Selection Rationale: Tree-based models can handle NaN natively (by sending them to a default branch), but for compatibility across all libraries, we impute with median. Infinite values are replaced with median to avoid library crashes.

13.2 Reproducibility

13.2.1 Random Seed Management

$$\text{set_seed}(\text{seed} = 42) \quad \text{for all stochastic components} \quad (103)$$

Enables deterministic model training and validation across runs.

Selection Rationale: Seed 42 is a common convention. All random operations (data shuffling, bagging, feature sampling) are seeded to ensure that the model results are reproducible. This is critical for debugging and compliance.

13.3 Model Serialization

Trained models are serialized for deployment:

$$\text{models} \rightarrow \text{.pkl} / \text{.joblib} / \text{.xgb files} \quad (104)$$

Facilitates inference without retraining.

Selection Rationale: Pickle is used for scikit-learn models, joblib for larger models (efficient compression), and XGBoost's native binary format for XGBoost models. This ensures fast loading in production.

14 Advanced Topics

14.1 Concept Drift Detection

Time-series data exhibits non-stationarity; demand patterns evolve over time.

14.1.1 Performance Degradation Monitoring

$$\text{drift_detected} = \mathbf{1}(R_t^2 - R_{t-\Delta t}^2 < -\theta) \quad (105)$$

where $\theta = 0.05$ is a degradation threshold.

Action: Trigger model retraining when drift is detected.

Selection Rationale: A drop of more than 0.05 in weekly R^2 (compared to the previous week) is considered significant. This threshold was chosen by analyzing historical performance fluctuations; smaller drops are within normal variation.

14.2 Hyperparameter Sensitivity Analysis

Sensitivity analysis quantifies the impact of hyperparameter changes.

14.2.1 Parameter Importance via Gradients

$$\text{sensitivity}_\lambda = \left| \frac{\partial R^2}{\partial \lambda} \right| \quad (106)$$

High sensitivity indicates critical hyperparameters warranting careful tuning.

Selection Rationale: We use finite differences to estimate sensitivity. Parameters like ‘num_leaves’ and ‘learning_rate’ typically show high sensitivity. This analysis guides where to focus man

A Detailed Mathematical Derivations

A.1 Derivation of the Target Encoding Smoothing Formula

Given a Bayesian model: $y_{ik} \sim N(\mu_k, \sigma^2)$, with prior $\mu_k \sim N(\mu_0, \tau^2)$. The posterior mean is:

$$\mathbb{E}[\mu_k | \mathbf{y}_k] = \frac{n_k \bar{y}_k / \sigma^2 + \mu_0 / \tau^2}{n_k / \sigma^2 + 1 / \tau^2}. \quad (107)$$

Set $\mu_0 = \bar{y}_{\text{global}}$, and define $m = \sigma^2 / \tau^2$. Then the posterior mean becomes:

$$\hat{\mu}_k = \frac{n_k \bar{y}_k + m \bar{y}_{\text{global}}}{n_k + m}. \quad (108)$$

This shows that m acts as a pseudo-count that regularizes the estimate.

A.2 Derivation of Ensemble Variance Reduction

For M independent models with equal variance σ^2 and pairwise correlation ρ , the variance of the average is:

$$\text{Var} \left(\frac{1}{M} \sum_{i=1}^M \hat{f}_i \right) = \frac{\sigma^2}{M} (1 + (M-1)\rho). \quad (109)$$

When models are uncorrelated ($\rho = 0$), variance reduces as σ^2/M . Our Optuna-weighted ensemble exploits low correlation among base models (LightGBM, XGBoost, CatBoost, etc.) to achieve substantial variance reduction.

A.3 Derivation of the Calibration Scale Factor

Suppose the true demand y has mean μ . The ensemble predictions $\hat{y}$ have mean $\hat{\mu}$. If we assume a multiplicative bias: $\mathbb{E}[y] = \beta \mathbb{E}[\hat{y}]$, then the unbiased estimator is $y_{\text{cal}} = \hat{y} \cdot (\mu / \hat{\mu})$. We also include a small boost factor $\alpha = 1.01$ to compensate for residual underprediction observed in validation.

B Summary Table: All Formulas

Category	Formula	Purpose
Temporal	$t_{\min} = 60h + m$	Convert time to minutes
Harmonic	$\sin_k = \sin(2\pi k t_{\min} / 1440)$	Periodic pattern
Encoding	$\hat{\mu}_k = \frac{n_k \bar{y}_k + m \bar{y}}{n_k + m}$	Target encoding
Loss	$\mathcal{L} = \frac{1}{n} \sum (y - \hat{y})^2$	MSE loss
Metric	$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}$	R^2 score
Ensemble	$\hat{y}_{\text{opt}} = \sum w_i \hat{y}_i$	Optuna blend
Bagging	$\hat{y}_{\text{bag}} = \frac{1}{S} \sum \hat{y}_s$	Seed bagging
Blend	$\hat{y}_{\text{blend}} = 0.7 \hat{y}_{\text{opt}} + 0.3 \hat{y}_{\text{bag}}$	Final pred.
Calibration	$\hat{y}_{\text{cal}} = \hat{y} \times \frac{\mu_{\text{exp}}}{\mu_{\text{OOF}}} \times 1.01$	Rescale mean
Residual	$\text{Bias} = \frac{1}{n} \sum (y_i - \hat{y}_i)$	Systematic error

C Hyperparameter Reference

Model	Key Parameters	Values	Notes
LightGBM	num_leaves, learning_rate	31, 0.05	Leaf-wise boosting
XGBoost	max_depth, reg_lambda	8, 1.0	2nd-order optimization
CatBoost	iterations, learning_rate	500, 0.05	Categorical native
ExtraTrees	n_estimators, max_depth	300, 20	Random splits
HistGB	learning_rate, max_iter	0.1, 200	Histogram binning

D Debugging Checklist

1. **High RMSE:** Check for missing data, outliers, or class imbalance
2. **Overfitting:** If train $R^2 \gg$ test R^2 , increase regularisation or reduce model capacity
3. **Underfitting:** If train $R^2 \approx$ test $R^2 \approx 0$, increase model complexity or feature engineering
4. **Data Leakage:** Verify target encoding is fold-safe; check lag features for future references
5. **NaN Predictions:** Investigate extreme feature values; apply clipping to outputs
6. **Slow Inference:** Profile feature computation; consider feature selection or dimensionality reduction

E Glossary

Geohash Compact geographic coordinate encoding

K-Means Unsupervised clustering partitioning data into k groups

Target Encoding Replacement of categorical features with target statistics

OOF Predictions Out-of-fold predictions from cross-validation

Ensemble Combination of multiple models

Calibration Post-hoc adjustment of predictions to match empirical statistics

RMSE Root mean squared error; metric in target units

R^2 Coefficient of determination; proportion of variance explained

Bias Systematic prediction error (mean of residuals)

Data Leakage Inadvertent use of validation/test data during training

Optuna Bayesian hyperparameter optimization framework

TPE Tree-structured Parzen Estimator